What is Software Tools and Process Automation?

- Software tools help people create, test, manage, and operate software.

- Process automation uses defined rules and tools to perform repeatable steps with less manual effort.

- Together, they support a repeatable path from an idea to a running application.

SIMPLE EXPLANATION

Tools support software work, while automation carries out repeatable steps. Together, they help a team turn an idea into a working application using a consistent process.

Why automation matters

- Repetitive manual steps take time and can be performed inconsistently.
- Automated builds and tests provide earlier feedback; version history makes changes traceable.
- Repeatable processes help teams collaborate, but still need human judgment, maintenance, and security.

SIMPLE EXPLANATION

Running the same checks automatically reduces repetitive work and gives feedback sooner. People still decide what to build, review results, and maintain the process.

Who uses software tools and automation?

- Developers and students track code and collaborate using version control.
- QA and test engineers automate checks and investigate quality problems.
- Operations, platform, and security teams automate environments, delivery, and monitoring.

SIMPLE EXPLANATION

Students, developers, testers, operations staff, and security teams all use these practices. Their responsibilities overlap, so effective automation depends on collaboration.

Example: a connected student-project workflow

- A developer changes a React task-management interface and records the change in Git.
- A CI workflow builds and tests it; the team reviews results before releasing.
- The team operates the application, monitors it, and uses feedback to improve the next change.

SIMPLE EXPLANATION

A student changes a React component, records the change in Git, and runs automated checks. After release, the team observes the application and uses that feedback to plan improvements.

Version control: definition and importance

- Version control records changes to files over time.
- Compare versions, recover earlier work, and understand who changed what.
- A shared history helps teams collaborate.

SIMPLE EXPLANATION

Version control keeps a history of file changes. Instead of guessing which copied folder is current, a team can compare versions, recover earlier work, and understand changes.

Centralized and distributed version control

- Centralized VCS: clients depend on a central version database.
- Distributed VCS: clones include repository history.
- Git is distributed; most local operations work offline.

SIMPLE EXPLANATION

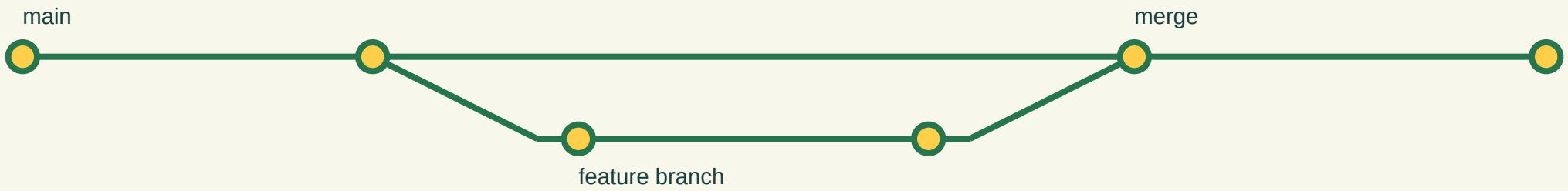
A centralized system keeps its version database on a central server. In a distributed system such as Git, clones include repository history, allowing many operations to work offline.

Git repositories, commits, branches, and remotes

- Repository: files and their recorded history. Commit: a recorded snapshot.

- Branch: a movable line of development. Merge: integrate histories.

- Clone copies a repository; push sends commits; pull fetches and integrates.



SIMPLE EXPLANATION

A repository stores a project and its history. Commits record snapshots, branches separate development, and merging brings histories together. Clone, push, and pull help exchange work with other repositories.

Basic Git workflow

- Modify your working files.
- Stage the changes you intend to include.
- Commit locally, then push to a shared remote.

```
Modify -> Stage -> Commit -> Push
```

SIMPLE EXPLANATION

First edit files, then stage the changes you want to record. Commit saves them in your local history; push shares the commits with a remote repository.

GitHub collaboration features

- Host repositories and discuss work through issues.
- Propose changes with pull requests and code review.
- GitHub Actions can automate builds, tests, and other workflows.

SIMPLE EXPLANATION

GitHub hosts Git repositories and adds tools for teamwork. Pull requests support review, issues track work, and Actions can run automated workflows such as builds and tests.

GitLab collaboration and CI/CD

- Host Git repositories and organize collaboration.
- Use merge requests to review and integrate changes.
- Define GitLab CI/CD jobs in .gitlab-ci.yml.

SIMPLE EXPLANATION

GitLab also hosts Git repositories. Teams review changes through merge requests and define automated jobs in a pipeline configuration that runners execute.

Git vs GitHub vs GitLab

- Git -> distributed version control software.
- GitHub -> Git hosting, pull requests, issues, and Actions.
- GitLab -> Git hosting, merge requests, and integrated CI/CD.

SIMPLE EXPLANATION

Git is the version control software. GitHub and GitLab are development platforms built around Git, adding hosting, review, and automation features.

Example: a React feature branch

- Clone an existing repository first.

- Create a feature branch, edit React components, and run checks.

- Commit and push, then open a pull or merge request.

```
git clone https://example.com/team/student-tasks.git
cd student-tasks
git switch -c feature-login
git add .
git commit -m "Add login feature"
git push -u origin feature-login
```

SIMPLE EXPLANATION

Create a separate branch for a login feature, edit the project, and run checks. Stage and commit the work, push the branch, then request review. Replace the example repository URL with a real one.

CI/CD overview

- CI integrates changes frequently with automated checks.
- Delivery keeps validated changes ready for production release.
- Deployment automatically releases eligible validated changes.

SIMPLE EXPLANATION

Continuous integration checks frequently integrated changes. Continuous delivery keeps validated software ready for release, while continuous deployment automatically releases eligible validated changes to production.

Continuous integration

- Developers integrate small changes frequently.

- An automated workflow builds and tests integrated changes.

- Failures give feedback so the team can fix problems promptly.

```
Source -> Build -> Test -> Release -> Deploy -> Monitor
```

SIMPLE EXPLANATION

Integrating small changes frequently helps teams discover integration problems earlier. Automated builds and tests provide feedback, but CI by itself does not mean the application is deployed.

Build, test, and integration checks

- Install reproducible dependencies and compile or bundle.
- Run appropriate unit, integration, and other checks.
- Review results and fix failures before proceeding.

SIMPLE EXPLANATION

The build prepares the application and tests check selected behaviors. A successful run means the configured checks passed; it cannot prove the software has no bugs.

Continuous delivery

- Automate validation and prepare deployable artifacts.
- Keep software in a releasable condition.
- Production release remains a deliberate approval or business decision.

SIMPLE EXPLANATION

Continuous delivery keeps a validated version ready for a production release. The team or business still makes the deliberate decision to release it.

Continuous deployment

- Eligible changes pass the required automated checks.
- The pipeline deploys them to production without a manual approval step.
- Monitoring and rollback strategies remain essential.

SIMPLE EXPLANATION

Continuous deployment sends eligible changes to production automatically after required checks pass. Monitoring and recovery controls are still necessary.

Delivery vs deployment

- Delivery -> validated artifact -> production approval -> deploy.
- Deployment -> validated artifact -> automatic production deploy.
- Both rely on automated validation and release readiness.

SIMPLE EXPLANATION

The key difference is the production approval gate. Delivery retains a deliberate release decision; deployment automates production release for eligible validated changes.

CI/CD pipeline stages

- Source -> build -> test -> release -> deploy -> monitor.

- A failed required test stops downstream release and deployment.

- This example includes deployment; many CI workflows stop after testing.

```
Source -> Build -> Test -> Release -> Deploy -> Monitor
```

SIMPLE EXPLANATION

This example moves from source through build, test, release, deployment, and monitoring. A failed required test blocks downstream release and deployment. Some CI workflows finish after validation.

CI/CD tools

- GitHub Actions: repository workflows defined in YAML.
- GitLab CI/CD: pipelines and jobs executed by runners.
- Jenkins: extensible automation server for build and delivery workflows.

SIMPLE EXPLANATION

GitHub Actions, GitLab CI/CD, and Jenkins can automate development workflows. The tool runs the configured work, while the team remains responsible for designing and maintaining it.

Example: GitHub Actions for React

- Run on pushes and pull requests.

- Install from the lockfile, test once, and build.

- This workflow validates code; it does not deploy.

```yaml
name: React CI
on: [push, pull_request]
jobs:
  validate:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm test -- --run
      - run: npm run build
```

SIMPLE EXPLANATION

On a push or pull request, this workflow installs locked dependencies, runs tests, and builds the React project. It assumes a package-lock.json and a Vitest test script. It validates code without deploying it.

CI/CD benefits, security, and limitations

- Repeatable checks reduce manual repetition and expose problems earlier.
- Protect secrets, limit token permissions, and review dependencies.
- Flaky tests, unsafe scripts, and weak coverage limit confidence.

SIMPLE EXPLANATION

Automation improves repeatability and shortens feedback loops. Teams must still protect secrets, limit permissions, review dependencies, and fix flaky or incomplete checks.

What is DevOps?

- Development and operations collaborate to deliver and run software.
- Culture, practices, and tools support faster, more reliable work.
- Automation supports people; it does not replace responsibility.

SIMPLE EXPLANATION

DevOps combines development and operations through collaboration, shared ownership, and supporting practices. It is a way of working rather than one particular tool.

Development and operations collaboration

- Development creates and improves application behavior.
- Operations supports availability, environments, and reliable running systems.
- Shared goals and feedback reduce handoff friction.

SIMPLE EXPLANATION

Developers create application behavior, while operations work supports reliable running systems. Sharing goals and feedback reduces problems caused by disconnected handoffs.

DevOps culture and shared responsibility

- Collaborate across disciplines.
- Share responsibility for quality, security, and production outcomes.
- Automate repeatable tasks and learn from incidents.

SIMPLE EXPLANATION

A DevOps team shares responsibility for quality, security, and production outcomes. Automation supports that collaboration, and incidents become opportunities to improve the system.

The eight-phase DevOps lifecycle

- Plan -> code -> build -> test.

- Release -> deploy -> operate -> monitor.

- Monitoring feeds the next plan. Security spans every phase.

```
Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor
```

SIMPLE EXPLANATION

Work moves through planning, coding, building, testing, releasing, deploying, operating, and monitoring. Monitoring informs the next plan. These eight phases are a teaching model; security spans the whole cycle.

Infrastructure as Code and configuration management

- Machine-readable configuration defines infrastructure.
- Version and review changes; use repeatable provisioning.
- Configuration management helps maintain desired system settings.

SIMPLE EXPLANATION

Infrastructure as Code describes infrastructure in machine-readable files that can be reviewed and versioned. Configuration management helps maintain intended system settings consistently.

Containers and deployment automation

- Docker packages applications and dependencies in container images.
- Kubernetes orchestrates containerized workloads.
- Automation can roll out updates, check health, and support recovery.

SIMPLE EXPLANATION

Docker packages applications and dependencies into container images. Kubernetes coordinates containerized workloads. These tools can support DevOps, but Kubernetes is not required for every team.

Monitoring, logging, security, and feedback

- Metrics show trends; logs record events; traces connect requests.
- Alerts should help people respond to actionable problems.
- Integrate security checks and operational feedback throughout delivery.

SIMPLE EXPLANATION

Metrics reveal trends, logs record events, and traces connect activity across requests. Teams use this evidence to investigate problems, improve operations, and feed learning back into development.

Siloed workflow vs DevOps collaboration

- Silos: large batches, separate goals, and late operational feedback.
- DevOps: small changes, shared goals, and ongoing collaboration.
- Practices must fit the team and its constraints.

SIMPLE EXPLANATION

Separate teams and large handoffs can delay useful feedback. DevOps practices encourage smaller changes, shared goals, and earlier collaboration, adapted to the team and its constraints.

DORA delivery performance metrics

- Change lead time: commit to production; deployment frequency: how often production deployments occur.

- Failed deployment recovery time: time to recover from a failed production deployment.

- Change fail rate: proportion requiring immediate intervention; deployment rework rate: unplanned deployments caused by production incidents.

SIMPLE EXPLANATION

Use the five metrics to understand how quickly and safely a service changes and how much deployment rework occurs. Compare the same service over time to guide improvement, rather than scoring individual developers.

How Git, CI/CD, and DevOps work together

- Git records the change and enables review.
- CI/CD validates and supports repeatable delivery.
- DevOps connects people, practices, operation, and learning.

SIMPLE EXPLANATION

Git records the change, CI/CD automates validation and delivery steps, and DevOps connects people with operational learning. They support different parts of the same software delivery process.

Example: Student Task Management System

- React interface -> GitHub repository and pull request.
- GitHub Actions validates tests/build; an approved release deploys to Vercel.
- Supabase provides application data; the team monitors errors and access policies.

SIMPLE EXPLANATION

In this illustrative setup, React provides the interface, GitHub stores changes, Actions runs checks, Vercel hosts the release, and Supabase supplies application data. The team must still review access, protect credentials, and monitor behavior.

DevOps challenges and good practices

- Challenges: siloed ownership, flaky tests, skill gaps, and legacy systems.
- Use small changes, code review, reproducible builds, and safe rollback.
- Choose useful measures and improve one bottleneck at a time.

SIMPLE EXPLANATION

Start with one practical improvement, such as fixing unreliable tests or making builds reproducible. Small changes, code review, safe recovery, and useful measurements help a team improve steadily.

References

Git / About version control

https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Git / Branching

https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

GitHub / Pull requests

https://docs.github.com/en/pull-requests

GitHub / Continuous integration

https://docs.github.com/en/actions/get-started/continuous-integration

References

GitLab / Merge requests

https://docs.gitlab.com/user/project/merge_requests/

GitLab / CI/CD pipelines

https://docs.gitlab.com/ci/pipelines/

AWS / Continuous delivery

https://aws.amazon.com/devops/continuous-delivery/

AWS / What is DevOps?

https://aws.amazon.com/devops/what-is-devops/

References

DORA / Five delivery performance metrics

https://dora.dev/guides/dora-metrics/

Docker / Containers

https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

Kubernetes / Overview

https://kubernetes.io/docs/concepts/overview/

Jenkins / Pipeline

https://www.jenkins.io/doc/book/pipeline/